Journal of Optimization Theory and Applications (2026) 211:21
https://doi.org/10.1007/s10957-026-03125-9

Disciplined Biconvex Programming

Hao Zhu[1] · **Joschka Boedecker**[1]

Received: 23 December 2025 / Accepted: 26 September 2026
© The Author(s) 2026

Abstract

We introduce *disciplined biconvex programming* (DBCP), a modeling framework for specifying and solving biconvex optimization problems. Biconvex optimization problems arise in various applications, including machine learning, signal processing, computational science, and control. Solving a biconvex optimization problem in practice usually involves heuristic methods based on alternate convex search (ACS), which iteratively optimizes over one block of variables while keeping the other fixed, so that the resulting subproblems are convex and can be efficiently solved. However, designing and implementing an ACS solver for a specific biconvex optimization problem usually requires significant effort from the user, which can be tedious and error-prone. DBCP extends the principles of disciplined convex programming to biconvex problems, allowing users to specify biconvex optimization problems in a natural way based on a small number of syntax rules. The resulting problem can then be automatically split and transformed into convex subproblems, for which a customized ACS solver is then generated and applied. DBCP allows users to quickly experiment with different biconvex problem formulations, without expertise in convex optimization. We implement DBCP in the open-source Python package `dbcp`, as an extension to the well known domain specific language CVXPY for convex optimization.

Keywords Convex optimization · Biconvex programming · Alternate convex search · Domain specific language

Mathematics Subject Classification 90C25 · 90C40 · 90C90

Communicated by Juan-Enrique Martinez Legaz.

✉ Hao Zhu
zhuh@cs.uni-freiburg.de

Joschka Boedecker
jboedeck@cs.uni-freiburg.de

[1] IMBIT//BrainLinks-BrainTools and Department of Computer Science, University of Freiburg, Freiburg, BW 79110, Germany

Published online: 14 October 2026

1 Introduction

We consider biconvex optimization problems, which involve optimizing a biconvex objective function subject to a biconvex set constraint. Informally, by saying that a set (or function) is biconvex, we mean that it is convex in each of two blocks of variables when the other block is fixed. Biconvex optimization problems arise in various applications, including machine learning [51, 66, 75], signal and image processing [16, 45], recommender systems [17, 40], control [38, 56, 68], and brain computer interfaces [21, 61].

Unlike convex optimization problems, biconvex optimization problems are in general nonconvex and can be very hard to solve. In fact, many of them have been shown to be NP-hard (see [73], [65], and [75] for some examples). Nevertheless, various heuristic methods based on *alternate convex search* (ACS) [18] exist for finding good solutions to many biconvex optimization problems in practice. The basic idea of ACS-type methods is to iteratively optimize over one block of variables while keeping the other block fixed, such that each subproblem is convex and can be efficiently solved. Although in the most general case, theoretical convergence of ACS-type methods is only guaranteed to stationary points, these methods turn out to work quite well in practice and hence have become the most popular choice for finding a satisfactory solution to biconvex optimization problems.

Designing and implementing an ACS solver for a specific biconvex optimization problem usually requires significant effort from the user, including properly partitioning the biconvex problem into convex subproblems, implementing robust and efficient solvers for the subproblems, and designing appropriate stopping criteria. Moreover, maintaining numerical stability and ensuring convergence of the ACS procedure typically necessitate integrating appropriate regularizations or modifications into the subproblems to ensure properties such as strong convexity and feasibility. With the help of *domain specific languages* (DSLs) for convex optimization, such as CVXPY [3, 19], specifying and solving the (modified) convex subproblems is largely simplified and automated, while the other steps still require the user to have expert knowledge about convex analysis and programming, and can be tedious and error-prone.

In this paper, we propose a modeling framework for biconvex optimization problems, named *disciplined biconvex programming* (DBCP), which extends the ideas of *disciplined convex programming* (DCP) [36] to biconvex problems. DBCP for biconvex optimization is analogous to DCP for convex optimization, which allows users to specify biconvex optimization problems in a natural way based on a small number of syntax rules. When an optimization problem description complies with the DBCP syntax rules, it is guaranteed to be a valid biconvex problem, and more importantly, it can be automatically split and transformed into convex (specifically, DCP-compliant) subproblems. A customized ACS solver with procedures for handling initialization and improving numerical stability is then generated and applied. Like DCP does for convex optimization, DBCP makes it easy to specify and solve biconvex problems, allowing users to quickly prototype and experiment with different biconvex problem formulations, without expertise in (or even knowledge of) convex analysis and solution methods for biconvex problems. In fact, most users might be unaware of the

variable partition, transformation, and solution processes, through which a solution to the biconvex problem is found. We implement DBCP in a Python package, named `dbcp`, as an extension of the DSL CVXPY, which is fully open-source and available at

https://github.com/dxogrp/dbcp.

1.1 Previous and Related Work

Biconvex analysis. The first recognition of biconvex structure in the context of mathematical programming can be traced back to Falk and Soland [22] in the 1960 s. Only a few papers exist in the literature where biconvex sets are investigated. Most of the corresponding theoretical results can be found in the papers of Aumann and Hart [7] and Goh *et al.* [32]. Biconvex functions appear regularly in practice and hence have been discussed widely in the literature. Properties of biconvex functions that are most relevant to optimization problems can be found in the works of Goh *et al.* [32] and Gorski *et al.* [33]. In addition, biconvex functions play an important role in martingale theory, and, in particular, in the analysis of Banach and Hilbert spaces [7, 12, 13, 44]. Biconvex functions can also be used to derive results on the robust stability of control systems in practical control engineering [30, 31]. Thibault [64], Jouak and Thibault [41], and Borwein [8] analyzed the continuity and differentiability of measurable biconvex operators in topological vector spaces. Al-Khayyal and Falk [5] published results on the maximization of biconvex functions. In the most general case, very little can be said about the global or even local optimality properties of biconvex optimization problems. Nevertheless, some useful properties of partial optimality conditions (*i.e.*, conditions for stationary points) of biconvex problems are discussed in [33, 74].

Solution methods for biconvex problems. Current solution methods for biconvex problems can be roughly categorized into two classes: heuristic methods and global optimization methods. Heuristic methods for biconvex problems aim at finding stationary points of the biconvex objective function, and are mostly based on the idea of alternately optimizing two convex subproblems, *i.e.*, the ACS-type methods. ACS methods are a special case of *block relaxation methods* [18, 54, 72] where the variables are divided into disjoint blocks, and in each step, only the variables of an active block are optimized while those of the other blocks are fixed. In particular, for ACS, only two blocks of variables defined by the convex subproblems are activated in cycles. Since the resulting subproblems are convex, efficient convex minimization methods can be used to solve these subproblems. A survey on ACS methods for biconvex optimization problems can be found in [74]. Gorski *et al.* [33] showed that under weak assumptions, all solution points generated by ACS form a compact connected set and that each of these points is a stationary point of the objective function. However, no better convergence results regarding local or global optimality properties can be obtained in general. Regarding the global optimization methods, Floudas and Visweswaran [25] adapted the idea of *branch-and-bound* [9, 42] to solve biconvex problems to global optimality. Detailed mathematical background and an outline of this algorithm are given in [23–26], and some basic convergence properties of this method are discussed in [33].

Biconvex optimization applications. The practical usefulness of biconvex programming in many applications is also increasingly well known. In machine learning, nonnegative matrix factorization [43, 51, 66], dictionary learning [4], and bilinear regression [17, 37, 40] are all well known biconvex optimization problems, which have been widely applied in representation learning, signal processing, and recommender systems. The very popular k-means clustering problem can also be formulated as a biconvex program [46, 62, 75]. Blind deconvolution problems [16, 45] are another important class of biconvex optimization problems for image processing and communication. Other studies such as Mishra *et al.* [48] and Fosson [27] explored biconvex structures in sparse learning and reweighted regression. Biconvex problems also appear when dealing with bilinear matrix inequalities for bilinear control system synthesis [38, 56, 68]. In economics, Schaller and Boyd [57] formulate the regularized maximum likelihood estimation of a price elasticity matrix with a low-rank-plus-diagonal structure as a biconvex problem in the two low rank factors. Finally, Dyrholm *et al.* [21] and Shi *et al.* [61] applied bilinear discriminant component analysis techniques to brain computer interfaces. We discuss some of these applications in more detail in Sect. 6.
Disciplined convex programming. DCP [36] is a modeling framework for convex optimization problems. If a mathematical optimization problem is written following the DCP syntax ruleset, it is guaranteed to be a valid convex program, and can then be automatically transformed into an appropriate canonical form [3] for a numerical convex optimization solver, such as OSQP [63], ECOS [20], SCS [50], and Clarabel [34]. Many DSLs for convex optimization, such as YALMIP [47], CVX [35], Convex.jl [67], CVXPY [19], and CVXR [28], are based on the DCP ruleset. Over the years, DCP has been extended to handle stochastic [6], convex-concave [59], geometric [2], quasiconvex [1], and saddle [58] problems. The most closely related extension of DCP to our work is *disciplined multi-convex programming* (DMCP) [60], which provides a DSL for specifying and solving multi-convex optimization problems via block relaxation methods. However, it seems to be no longer actively supported or maintained. In this paper, we work with CVXPY and focus on biconvex problems only, since most multi-convex problems in practice can be formulated as biconvex problems by grouping variables appropriately. *Disciplined nonlinear programming* (DNLP) [15] is a recent extension of CVXPY for specifying and solving general nonlinear programs, including some biconvex problems. For more detailed comparisons of the proposed DBCP framework with DMCP and DNLP, see Sect. 7.2 and Sect. 7.3, respectively.

1.2 Outline

This paper is *not* about establishing new theoretical results or computational methods for biconvex optimization problems. Instead, we collect well known ideas and assemble them into a DSL for specifying and solving biconvex optimization problems in a natural human-readable way which is close to the mathematical formulation. These ideas are implemented in the open-source Python package `dbcp`, which extends CVXPY to support biconvex programming.

The rest of this paper is organized as follows. In Sect. 2, we provide formal definitions for biconvex sets, functions, and optimization problems, including some specific

examples of these objects, and briefly review their basic properties. In Sect. 3, we introduce the ACS-based method for solving biconvex optimization problems, and discuss some practical augmentations for handling initialization and improving numerical stability, which are all integrated into the `dbcp` package. Then in Sect. 4, we introduce the DBCP biconvex syntax ruleset for specifying biconvex optimization problems, as an extension of the DCP ruleset. The implementation and basic usage of the provided Python package `dbcp`, where the functions and features mentioned above are implemented, are discussed in Sect. 5. Finally, in Sect. 6, we present some specific numerical examples for specifying and solving biconvex optimization problems that frequently appear in practice using the `dbcp` package, to show the simplicity and effectiveness of our framework. These examples along with the corresponding code snippets also serve as a preliminary tutorial for new users to get started with `dbcp`.

2 Biconvex Programming

In this section, we provide formal definitions for biconvex sets, biconvex functions, and biconvex optimization problems to establish our notation, and we briefly review some of the basic properties of these objects. To start with, let $\mathcal{X} \subseteq \mathbf{R}^n$ and $\mathcal{Y} \subseteq \mathbf{R}^k$ be two nonempty closed convex sets. We should also note that throughout this paper, we use the convention that functions are implicitly extended-valued, *i.e.*, by writing $f : A \to \mathbf{R}$, we really mean $f : A \to \mathbf{R} \cup \{\infty\}$. In other words, a function defined on A can take the value ∞ for some points in A [11, 55].

2.1 Biconvex Sets

A set $B \subseteq \mathcal{X} \times \mathcal{Y} \subseteq \mathbf{R}^{n+k}$ is a *biconvex set* if for every fixed $y \in \mathcal{Y}$, the set

$$B_y = \{x \in \mathcal{X} \mid (x, y) \in B\} \subseteq \mathbf{R}^n$$

is convex, and for every fixed $x \in \mathcal{X}$, the set

$$B_x = \{y \in \mathcal{Y} \mid (x, y) \in B\} \subseteq \mathbf{R}^k$$

is convex.

Obviously, a biconvex set is not necessarily convex in general. Figure 1 shows some examples of biconvex sets which are not convex. In particular, a biconvex set does not even have to be connected, as shown by the example

$$B = \{(x, y) \in \mathbf{R}^2 \mid x, y < 0\} \cup \{(x, y) \in \mathbf{R}^2 \mid x, y > 0\},$$

whose picture is shown on the right of Fig. 1.

We should note that whenever several biconvex sets, biconvex functions, or biaffine mappings are considered together in the properties below, they are assumed to share the same fixed partition of variables into the x- and y-blocks. An important property

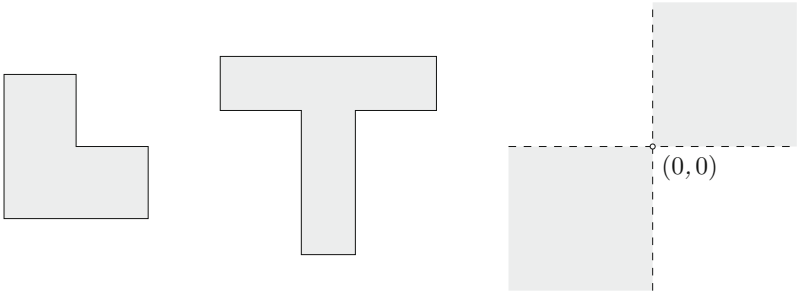


Fig. 1 Examples of biconvex sets

of biconvex sets is that, as with convex sets, the intersection of an arbitrary collection of biconvex sets is still a biconvex set [7, 33].

2.2 Biconvex Functions

A function $f : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$ is a *biconvex function* if its effective domain

$$\mathbf{dom}\ f = \{(x, y) \in \mathcal{X} \times \mathcal{Y} \mid f(x, y) < \infty\}$$

is a biconvex set, and for every fixed $y \in \mathcal{Y}$, the function

$$f_y : \mathcal{X} \to \mathbf{R}, \quad x \mapsto f(x, y),$$

is convex in x, and for every fixed $x \in \mathcal{X}$, the function

$$f_x : \mathcal{Y} \to \mathbf{R}, \quad y \mapsto f(x, y),$$

is convex in y. In other words, a biconvex function is one that is convex in each of two blocks of variables when the other block is fixed. We can also define *biconcave*, *biaffine*, and *bilinear* functions similarly, by replacing the property of being convex for f_y and f_x by the property of being concave, affine, or linear, respectively. Figure 2 shows an example of a biconvex function given by $f(x, y) = x^2 y^2$.

Now we list some basic properties of biconvex functions. We do not provide formal proofs here; interested readers may refer to [32] and [33] for the corresponding proofs and discussion of the following statements (which are more or less obvious with the basic convex analysis toolbox [55]). Similar to convex functions, the sublevel sets of a biconvex function $f : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, given by

$$C_\alpha = \{(x, y) \in \mathbf{dom}\ f \mid f(x, y) \leq \alpha\},$$

are biconvex sets, for any $\alpha \in \mathbf{R}$. Also, many arithmetic properties that are valid for convex functions can be transferred to the biconvex case:

– *Nonnegative weighted sums.* Evidently, if f is a biconvex function and $w \geq 0$, then the function wf is biconvex. If f_1 and f_2 are both biconvex functions, then

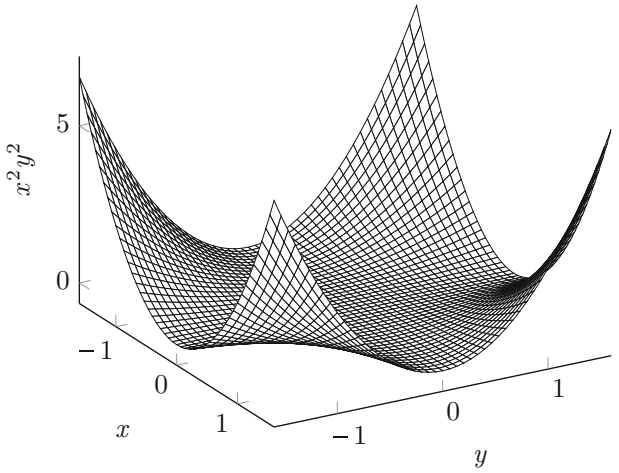


Fig. 2 Graph of the biconvex function $f(x, y) = x^2y^2$

so is their sum $f_1 + f_2$. In the general case, if $f_1, \ldots, f_m$ are biconvex functions, and $w_1, \ldots, w_m \geq 0$, then the function

$$f = w_1 f_1 + \cdots + w_m f_m$$

is also biconvex.

– *Composition with a biaffine mapping.* If $h : \mathbf{R}^m \to \mathbf{R}$ is convex, and $g : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}^m$ is biaffine, then the function $f : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, given by $f(x, y) = h(g(x, y))$, is biconvex. In particular, if $h : \mathbf{R} \to \mathbf{R}$ is convex, and $A \in \mathbf{R}^{m\times n}$, $C \in \mathbf{R}^{m\times k}$, $b, d \in \mathbf{R}^m$, then the function $f : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, given by

$$f(x, y) = h((Ax + b)^T (Cy + d))$$

is biconvex.

– *Pointwise maximum and supremum.* If $f_1, \ldots, f_m$ are biconvex functions, then the function

$$f(x, y) = \max\{f_1(x, y), \ldots, f_m(x, y)\}$$

is also biconvex. More generally, if $\{f_i\}_{i\in I}$ is a family of biconvex functions indexed by a set I, then the function

$$f(x, y) = \sup_{i\in I} f_i(x, y)$$

is biconvex.

– *Composition.* If $h : \mathbf{R} \to \mathbf{R}$ is convex and nondecreasing, and $g : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$ is biconvex, then the function $f : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, given by $f(x, y) = h(g(x, y))$, is biconvex. This property can be easily extended to multivariate functions $h : \mathbf{R}^m \to \mathbf{R}$ that are convex and nondecreasing in each argument. Interested readers may refer to [11, §3.2.4] for more details.

We should note that the property of *composition with a biaffine mapping* above is a special case of the more general *composition* property, but is of more practical importance, since most biconvexity structures in applications arise from composing convex functions with biaffine mappings. (See Sect. 6 for some examples.)

2.3 Biconvex Optimization Problems

A *biconvex optimization problem* is one of the form

$$\begin{array}{ll} \text{minimize} & f_0(x, y) \\ \text{subject to} & f_i(x, y) \leq 0, \quad i = 1, \ldots, m \\ & h_i(x, y) = 0, \quad i = 1, \ldots, p, \end{array} \tag{2.1}$$

where $x \in \mathcal{X}$, $y \in \mathcal{Y}$ are the optimization variables. The functions $f_i : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, $i = 0, \ldots, m$, are biconvex, and $h_i : \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, $i = 1, \ldots, p$, are biaffine. Here (and also in Sect. 3), when discussing theoretical properties, we assume that the problem variables are partitioned into two disjoint and exhaustive blocks, x and y, for notational convenience and in accordance with the convention in the literature. However, in practice, the two groups passed to our package interface must remain disjoint but need not be exhaustive; see Sect. 5 for more details. Roughly speaking, the problem (2.1) can be interpreted as minimizing a biconvex objective function over a biconvex feasible set defined by biconvex inequality constraints and biaffine equality constraints, since each of these constraints defines a biconvex set, and the intersection of biconvex sets is still a biconvex set (*cf.* Sect. 2.1). The *feasible set* of (2.1) is defined as

$$\mathcal{D} = \left\{ (x, y) \in \mathcal{X} \times \mathcal{Y} \;\middle|\; \begin{array}{l} f_0(x, y) < \infty \\ f_i(x, y) \leq 0, \quad i = 1, \ldots, m \\ h_i(x, y) = 0, \quad i = 1, \ldots, p \end{array} \right\}.$$

Different from convex optimization problems, in the most general case, very little can be said about the global or even local optimality properties of biconvex optimization problems. Instead, *partial optimality* is usually considered for biconvex optimization problems, which is even weaker than local optimality. Suppose $(x^\star, y^\star) \in \mathcal{D}$ is a feasible point of (2.1), then $(x^\star, y^\star)$ is a *partially optimal point* of (2.1) if

$$f_0(x^\star, y^\star) \leq f_0(x, y^\star)$$

for all $x \in \mathcal{D}_{y^\star}$, and

$$f_0(x^\star, y^\star) \leq f_0(x^\star, y)$$

for all $y \in \mathcal{D}_{x^\star}$, where

$$\mathcal{D}_{y^\star} = \left\{ x \in \mathcal{X} \;\middle|\; \begin{array}{l} f_0(x, y^\star) < \infty \\ f_i(x, y^\star) \leq 0, \quad i = 1, \ldots, m \\ h_i(x, y^\star) = 0, \quad i = 1, \ldots, p \end{array} \right\},$$

and

$$\mathcal{D}_{x^\star} = \left\{ y \in \mathcal{Y} \;\middle|\; \begin{array}{ll} f_0(x^\star, y) < \infty & \\ f_i(x^\star, y) \leq 0, & i = 1, \ldots, m \\ h_i(x^\star, y) = 0, & i = 1, \ldots, p \end{array} \right\}.$$

It can be shown that for a differentiable biconvex optimization problem, every stationary point of f_0 over $\mathcal{D}$ is partially optimal, and vice versa [33]. However, such a point is not necessarily a local optimum, as stationary points can be saddle points of the objective function. Some necessary conditions for a partially optimal point of a biconvex problem to be a local optimum are discussed in [33], but in general, no stronger results can be obtained. Although partial optimality is a weak notion of optimality, partially optimal points for biconvex problems are still widely used in practice, and often turn out to be good enough for many applications.

3 Solving Biconvex Problems

As a result of the optimality properties regarding biconvex optimization problems discussed in Sect. 2.3, 'solving' a biconvex optimization problem in practice usually amounts to finding a stationary point of the objective function over the feasible set. In this section, we introduce the ACS heuristic for this purpose that is implemented in the DBCP framework, where the basic idea is to transform the biconvex problem into two convex subproblems, which can then be handled directly by DCP-type DSLs for convex optimization. We also discuss some practical augmentations to ACS when dealing with the biconvex problem initialization and numerical stability issues.

3.1 Alternate Convex Search

ACS as a minimization method is a special case of block relaxation methods [18], where the variables are divided into two disjoint blocks, and in each step, only the variables of an active block are optimized while those of the other block are fixed. Specifically, for a biconvex optimization problem of the form (2.1), ACS iterates between solving the following two convex subproblems:

$$\begin{array}{ll} \text{minimize} & f_0(x, \tilde{y}) \\ \text{subject to} & f_i(x, \tilde{y}) \leq 0, \quad i = 1, \ldots, m \\ & h_i(x, \tilde{y}) = 0, \quad i = 1, \ldots, p, \end{array} \tag{3.1}$$

where $x \in \mathcal{X}$ is the variable, $\tilde{y} \in \mathcal{Y}$ is a fixed value of y, and

$$\begin{array}{ll} \text{minimize} & f_0(\tilde{x}, y) \\ \text{subject to} & f_i(\tilde{x}, y) \leq 0, \quad i = 1, \ldots, m \\ & h_i(\tilde{x}, y) = 0, \quad i = 1, \ldots, p, \end{array} \tag{3.2}$$

where $y \in \mathcal{Y}$ is the variable, and $\tilde{x} \in \mathcal{X}$ is a fixed value of x. Since the problems (3.1) and (3.2) are convex, efficient convex minimization methods can be used to solve these subproblems. The full ACS procedure is summarized in the following algorithm.

Algorithm 3.1 ALTERNATE CONVEX SEARCH.

given a starting point $(x^{(0)}, y^{(0)}) \in \mathcal{D}$.
$k := 0$.
repeat
1. *Solve (3.1) with* $\tilde{y} = y^{(k)}$. Try to obtain

$$x^{(k+1)} := \underset{x \in \mathcal{X}}{\operatorname{argmin}} \left\{ f_0(x, y^{(k)}) \;\middle|\; \begin{array}{l} f_i(x, y^{(k)}) \le 0, \; i = 1, \ldots, m \\ h_i(x, y^{(k)}) = 0, \; i = 1, \ldots, p \end{array} \right\}, \tag{3.3}$$

   and quit if infeasible.
2. *Solve (3.2) with* $\tilde{x} = x^{(k+1)}$. Try to obtain

$$y^{(k+1)} := \underset{y \in \mathcal{Y}}{\operatorname{argmin}} \left\{ f_0(x^{(k+1)}, y) \;\middle|\; \begin{array}{l} f_i(x^{(k+1)}, y) \le 0, \; i = 1, \ldots, m \\ h_i(x^{(k+1)}, y) = 0, \; i = 1, \ldots, p \end{array} \right\}, \tag{3.4}$$

   and quit if infeasible.
3. $k := k + 1$.

until stopping criteria are satisfied.

Gorski *et al.* [33] showed that under weak assumptions, the sequence of objective function values $\{f_0(x^{(k)}, y^{(k)})\}_{k=0}^{\infty}$ generated by ACS is monotonically nonincreasing and convergent. Furthermore, if the sequence $\{(x^{(k)}, y^{(k)})\}_{k=0}^{\infty}$ converges to $(x^\star, y^\star)$, then $(x^\star, y^\star)$ is a stationary point of f_0 (and hence is a partially optimal point of (2.1)). We should note that ACS is sensitive to initialization, *i.e.*, with different initial points $(x^{(0)}, y^{(0)})$, the sequence $\{(x^{(k)}, y^{(k)})\}_{k=0}^{\infty}$ generated through algorithm 3.1 may converge to different stationary points of f_0 with (probably) different function values.

There are several ways to define the stopping criteria for ACS. A simple choice is to stop when the difference between the objective values of (2.1) obtained in two consecutive iterations is below a certain threshold $\epsilon > 0$, *i.e.*, quit when

$$|f_0(x^{(k+1)}, y^{(k+1)}) - f_0(x^{(k)}, y^{(k)})| < \epsilon. \tag{3.5}$$

As a small variation, one may also use the difference between the optimal values of the problems (3.1) and (3.2) in one iteration as the criterion, *i.e.*, quit when

$$|f_0(x^{(k+1)}, y^{(k+1)}) - f_0(x^{(k+1)}, y^{(k)})| < \epsilon. \tag{3.6}$$

In practice, there is not much difference between using (3.5) and (3.6) as the termination criterion for the ACS procedure, except that using the latter does not require storing the objective value of (2.1) from the previous iteration. Note that when the objective values vary substantially in scale, it is useful in practice to combine the absolute threshold with a relative tolerance (see Sect. 5 for this implementation). Other choices include limiting the maximum number of iterations, or stopping when the changes in the optimization variables are below certain thresholds, *e.g.*, quit when

$$\max\{\|x^{(k+1)} - x^{(k)}\|_2, \; \|y^{(k+1)} - y^{(k)}\|_2\} < \epsilon,$$

for some $\epsilon > 0$.

3.2 Proximal Regularizations

Directly solving the subproblems (3.1) and (3.2) in the algorithm 3.1 may lead to numerical problems or slow convergence in practice. One possible reason for this is that the biconvex objective function f_0 can be very 'flat' in some region along certain directions when one block of variables is fixed (an example is shown in Fig. 2, in the region where x and y are close to zero), where the convex solver for subproblems may have difficulty making sufficient progress in minimizing the objective along the descent direction. A common technique to alleviate these issues is to add proximal regularization terms to the objective functions of the subproblems. Specifically, in the $(k+1)$th iteration of algorithm 3.1, instead of performing (3.3) and (3.4), we try to perform the following updates:

$$\begin{array}{rl} x^{(k+1)} := \operatorname{argmin}_{x\in\mathcal{X}} & f_0(x, y^{(k)}) + \lambda\|x - x^{(k)}\|_2^2 \\ \text{subject to} & f_i(x, y^{(k)}) \le 0, \quad i = 1, \ldots, m \\ & h_i(x, y^{(k)}) = 0, \quad i = 1, \ldots, p, \end{array} \tag{3.7}$$

and

$$\begin{array}{rl} y^{(k+1)} := \operatorname{argmin}_{y\in\mathcal{Y}} & f_0(x^{(k+1)}, y) + \lambda\|y - y^{(k)}\|_2^2 \\ \text{subject to} & f_i(x^{(k+1)}, y) \le 0, \quad i = 1, \ldots, m \\ & h_i(x^{(k+1)}, y) = 0, \quad i = 1, \ldots, p, \end{array} \tag{3.8}$$

where $\lambda \ge 0$ is the regularization parameter. When $\lambda = 0$, the updates (3.7) and (3.8) reduce to (3.3) and (3.4), respectively. When $\lambda > 0$, the proximal terms in (3.7) and (3.8) can be interpreted as adding soft trust region constraints to the respective optimization variables, which penalize large changes in the variables between two consecutive iterations, and in particular, the problems in (3.7) and (3.8) become strongly convex. In practice, a large value of λ helps improve numerical stability when solving the convex subproblems [10, 52]. It is also observed that adding the proximal regularizers can sometimes lead to better final points, *i.e.*, points with lower objective values [60], compared to those from the original ACS procedure. One possible intuition for this observation is that the proximal terms damp large changes in flat directions and favor nearby points when a subproblem has multiple or nearly equivalent solutions. This changes the trajectory of the nonconvex ACS procedure and can lead to a different final point, although sensitivity to initialization remains.

3.3 Initialization

Note that algorithm 3.1 requires a feasible starting point $(x^{(0)}, y^{(0)}) \in \mathcal{D}$, since otherwise, one of the subproblems (3.1) or (3.2) may be infeasible right at the first iteration. Formally, the corresponding feasibility problem for a biconvex problem of the form (2.1) can be written as

$$\begin{array}{ll} \text{find} & (x, y) \\ \text{subject to} & f_i(x, y) \le 0, \quad i = 1, \ldots, m \\ & h_i(x, y) = 0, \quad i = 1, \ldots, p \end{array} \tag{3.9}$$

with variables $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. Solving (3.9) directly can be as hard as solving the original biconvex problem (2.1), and, actually, it can even be NP-hard [65]. Here, we consider the following heuristic via relaxation to find a feasible starting point:

$$\begin{array}{ll} \text{minimize} & \mathbf{1}^T s + \|t\|_1 \\ \text{subject to} & s \succeq 0 \\ & f_i(x, y) \leq s_i, \quad i = 1, \ldots, m \\ & h_i(x, y) = t_i, \quad i = 1, \ldots, p, \end{array} \tag{3.10}$$

where $x \in \mathcal{X}$, $y \in \mathcal{Y}$ are the optimization variables, and $s \in \mathbf{R}^m$ and $t \in \mathbf{R}^p$ are the slack variables to relax the constraints in (3.9). The subscripts i of s_i and t_i denote the ith entry of the vectors s and t, respectively. Note that the relaxed biconvex feasibility problem (3.10) is always feasible, since by choosing sufficiently large s and t, all constraints can be satisfied. To solve (3.10), we can again use the ACS procedure introduced in Sect. 3.1. The full algorithm is given below.

Algorithm 3.2 FINDING A FEASIBLE STARTING POINT.

given a starting point $(x^{(0)}, y^{(0)}) \in \mathcal{X} \times \mathcal{Y}$.
$k := 0$.
repeat

1. $(x^{(k+1)}, s^\star, t^\star) := \underset{\substack{x \in \mathcal{X} \\ s \in \mathbf{R}^m, t \in \mathbf{R}^p}}{\operatorname{argmin}} \left\{ \mathbf{1}^T s + \|t\|_1 \;\middle|\; \begin{array}{l} s \succeq 0 \\ f_i(x, y^{(k)}) \leq s_i, \; i = 1, \ldots, m \\ h_i(x, y^{(k)}) = t_i, \; i = 1, \ldots, p \end{array} \right\}$.
   quit with $(x^{(k+1)}, y^{(k)})$ **if** $\mathbf{1}^T s^\star + \|t^\star\|_1 = 0$.
2. $(y^{(k+1)}, s^\star, t^\star) := \underset{\substack{y \in \mathcal{Y} \\ s \in \mathbf{R}^m, t \in \mathbf{R}^p}}{\operatorname{argmin}} \left\{ \mathbf{1}^T s + \|t\|_1 \;\middle|\; \begin{array}{l} s \succeq 0 \\ f_i(x^{(k+1)}, y) \leq s_i, \; i = 1, \ldots, m \\ h_i(x^{(k+1)}, y) = t_i, \; i = 1, \ldots, p \end{array} \right\}$.
   quit with $(x^{(k+1)}, y^{(k+1)})$ **if** $\mathbf{1}^T s^\star + \|t^\star\|_1 = 0$.
3. $k := k + 1$.

until maximum iterations are reached.

Unlike algorithm 3.1, the initialization procedure in algorithm 3.2 requires only a point in $\mathcal{X} \times \mathcal{Y}$, which is usually easy to obtain. If algorithm 3.2 quits with $\mathbf{1}^T s^\star + \|t^\star\|_1 = 0$ in some iteration, then the returned point is a feasible point of the original biconvex problem (2.1), which can then be used as a starting point for the ACS procedure in algorithm 3.1. However, we must note that there is no guarantee that the algorithm 3.2 will find a feasible point for any instance of the biconvex problem (2.1), even if such a point exists. In practice, as a generic practical method, this approach seems to work quite well.

3.4 Penalty Method and Infeasible Start

Now we integrate the relaxation, which transforms the biconvex feasibility problem (3.9) into (3.10), directly into the original biconvex problem (2.1), so that the ACS procedure can be applied even when starting from an infeasible point. Let $s \in \mathbf{R}^m$ and $t \in \mathbf{R}^p$ be slack variables to relax the inequality and equality constraints of (2.1),

respectively. We consider the following relaxed biconvex optimization problem:

$$\begin{array}{ll} \text{minimize} & f_0(x, y) + \nu(\mathbf{1}^T s + \|t\|_1) \\ \text{subject to} & s \succeq 0 \\ & f_i(x, y) \leq s_i, \quad i = 1, \ldots, m \\ & h_i(x, y) = t_i, \quad i = 1, \ldots, p, \end{array} \tag{3.11}$$

where $\nu > 0$ is a penalty parameter. The problem (3.11) is always feasible, since by choosing sufficiently large s and t, all constraints can be satisfied. Moreover, if the original biconvex problem (2.1) is feasible, then for sufficiently large ν, applying ACS to (3.11) will yield a final point $(x^\star, y^\star, s^\star, t^\star)$, such that $\mathbf{1}^T s^\star + \|t^\star\|_1 = 0$, *i.e.*, $(x^\star, y^\star)$ is feasible and partially optimal for (2.1) [49, 60]. The full algorithm is given as follows.

Algorithm 3.3 PENALTY ALTERNATE CONVEX SEARCH.

given a starting point $(x^{(0)}, y^{(0)}) \in \mathcal{X} \times \mathcal{Y}$ and sufficiently large $\nu > 0$.
$k := 0$.
repeat

1. $x^{(k+1)} := \underset{x \in \mathcal{X}}{\operatorname{argmin}} \left\{ \begin{array}{l|l} f_0(x, y^{(k)}) & s \in \mathbf{R}^m, \quad t \in \mathbf{R}^p, \quad s \succeq 0 \\ \quad +\nu(\mathbf{1}^T s + \|t\|_1) & f_i(x, y^{(k)}) \leq s_i, \quad i = 1, \ldots, m \\ & h_i(x, y^{(k)}) = t_i, \quad i = 1, \ldots, p \end{array} \right\}.$
2. $y^{(k+1)} := \underset{y \in \mathcal{Y}}{\operatorname{argmin}} \left\{ \begin{array}{l|l} f_0(x^{(k+1)}, y) & s \in \mathbf{R}^m, \quad t \in \mathbf{R}^p, \quad s \succeq 0 \\ \quad +\nu(\mathbf{1}^T s + \|t\|_1) & f_i(x^{(k+1)}, y) \leq s_i, \quad i = 1, \ldots, m \\ & h_i(x^{(k+1)}, y) = t_i, \quad i = 1, \ldots, p \end{array} \right\}.$
3. $k := k + 1$.

until stopping criteria are satisfied.

Compared to algorithm 3.1, the penalty ACS procedure in algorithm 3.3 can start from any point in $\mathcal{X} \times \mathcal{Y}$, and hence, the initialization step via, *e.g.*, algorithm 3.2, can be avoided. The same termination criteria as those discussed in Sect. 3.1 can still be used for algorithm 3.3. However, we must note that there is no guarantee that the final point returned by algorithm 3.3 is feasible for the original biconvex problem (2.1), even if such a point exists, since the penalty parameter ν may not be sufficiently large. In practice, the value of ν can be selected in an ad hoc manner, *i.e.*, one may try to increase ν and resolve (3.11) if the final point returned by algorithm 3.3 is still infeasible for (2.1). Finally, the proximal regularizations as in (3.7) and (3.8) can be readily integrated into the subproblems in the algorithm 3.3 to improve numerical stability.

4 Disciplined Biconvex Programming

We now present the DBCP biconvex ruleset for modeling biconvex optimization problems in a way that the biconvexity is easily verified by construction.

DCP ruleset. The DBCP ruleset is heavily based on the DCP ruleset, which consists of a library of convex atomic functions and a convex syntax ruleset that prescribes how these atomic functions may be composed to express (more complex) convex

optimization problems [36]. Specifically, all functions in a DCP-compliant problem must be formed as an *expression* consisting of variables, constants or parameters, and atomic functions. The sign and curvature of each DCP expression can be determined recursively using the signs and curvatures of its arguments together with the sign, curvature, and possibly sign-dependent monotonicity of the atomic function applied at each step. Based on this information, the convexity of the overall problem can be verified by checking whether each function in the problem satisfies the DCP composition rules [11, 36]. We make an observation that is critical for extending DCP to DBCP: The basic arithmetic properties, in particular, the composition property, of biconvex functions, as discussed in Sect. 2.2, are compatible with the DCP ruleset. In other words, an optimization problem formed by verifiable biconvex expressions is still biconvex if it complies with a variant of the DCP ruleset in which 'convex' is replaced by 'biconvex'.

DBCP product rule. To construct disciplined biconvex expressions, we first note that the product of expressions that are both nonconstant is prohibited in DCP, since the convexity of such expressions cannot be determined in general [36]; however, most biconvex expressions that appear in practice are constructed through variable multiplications. Hence, we introduce the following product rule for DBCP:

- Products involving a constant or parameter factor are valid in DBCP whenever they satisfy the DCP rules. If both the left-hand and right-hand expressions contain variables, a valid DBCP biconvex product expression should be one of the following forms:

$$\begin{gathered} \textit{affine} * \textit{affine} \\ \textit{affine-nonneg} * \textit{convex} \quad \text{or} \quad \textit{affine-nonpos} * \textit{concave} \\ \textit{convex-nonneg} * \textit{convex-nonneg} \quad \text{or} \quad \textit{concave-nonpos} * \textit{concave-nonpos} \end{gathered}$$

  The *nonneg* and *nonpos* qualifiers indicate that the expression is known to be nonnegative or nonpositive, respectively.
- Across the objective and constraints, the same two-block variable assignment must be used for every product in which both factors contain variables. For each such product, one factor must contain only variables from one block and the other only variables from the other block.

The common two-block assignment requirement in the last bullet prevents expressions like `x * y`, `y * z`, and `z * x` from appearing simultaneously in the same optimization problem, since `x * y` and `y * z` require `x` and `z` to belong to the same block, whereas `z * x` requires them to belong to different blocks. According to the definition and basic properties of biconvex functions listed in Sect. 2.2, expressions formed according to the above DBCP product rule are guaranteed to be biconvex.

5 Implementation

In this section, we introduce our Python implementation of the DBCP framework, as an extension to CVXPY [3, 19]. The corresponding open-source package, named `dbcp`, is available at

https://github.com/dxogrp/dbcp.

Biconvex atoms. Biconvex expressions in `dbcp` are created from fundamental atomic functions. Most of the atoms used in `dbcp` are inherited from CVXPY, such as inner product and elementwise multiplication. An example of the extended atoms that are not supported by CVXPY is the one-dimensional discrete *convolution* operation between two one-dimensional nonconstant expressions, which can be called by the user via `dbcp.convolve`. It has the same behavior as the `convolve` function in CVXPY, except that the first expression need not be a constant. The biconvex atom library is extensible, so that additional atoms can be added as necessary.
Specifying biconvex problems. Users can define their optimization variables, objective functions, and constraints using the standard CVXPY syntax, and a biconvex problem is constructed using:

```
prob = BiconvexProblem(obj, x_var, y_var, constr)
```

The argument `obj` is the objective constructed from a DBCP-compliant biconvex expression, `x_var` and `y_var` are iterables of the biconvex optimization variables, and `constr` is a list of DBCP-compliant biconvex constraints. The arguments `x_var` and `y_var` define the two disjoint groups of variables that are fixed on alternating steps during the ACS procedure, so that the two subproblems are convex. Note that `x_var` and `y_var` usually contain all the problem variables as in (2.1), but this is not necessarily the case; our implementation allows for more flexibility in defining these groups. Specifically, problem variables omitted from both `x_var` and `y_var` can remain active in both subproblems, provided both resulting subproblems are DCP-compliant. For example, Z is such a variable in the problem

$$\begin{array}{l}\text{minimize } \|XY + Z - A\|_F \\ \text{subject to } \|Z\|_F \leq 1,\end{array}$$

where $X \in \mathbf{R}^{m\times k}$, $Y \in \mathbf{R}^{k\times n}$, $Z \in \mathbf{R}^{m\times n}$ are optimization variables, and $A \in \mathbf{R}^{m\times n}$ is problem data. To specify this biconvex problem, one may use the following code:

```
import cvxpy as cp
from dbcp import BiconvexProblem

X = cp.Variable((m, k))
Y = cp.Variable((k, n))
Z = cp.Variable((m, n))

obj = cp.Minimize(cp.norm(X @ Y + Z - A, 'fro'))
constr = [cp.norm(Z, 'fro') <= 1]
prob = BiconvexProblem(obj, [X], [Y], constr)
```

Solving a biconvex problem. To solve the specified biconvex problem `prob`, one may simply call `prob.solve()`. There are many optional arguments that can be passed to `prob.solve()` to customize the solution procedure, most of which are directly inherited from CVXPY for the configuration of the convex solvers used to solve the subproblems. In addition to these standard arguments, `dbcp` also provides several specific options for controlling the ACS procedure. One of the most important ones is

`lbd`, which sets a value for the proximal regularization parameter λ in (3.7) and (3.8). When `lbd` is zero, no proximal regularization is added, *i.e.*, the original ACS procedure (algorithm 3.1) is used; if `lbd` is positive, then the proximal regularized updates in (3.7) and (3.8) are used instead of the original updates (3.3) and (3.4). It is recommended that the user specify (feasible) initial values for all optimization variables before calling `prob.solve()`. Otherwise, `dbcp` will try to generate random initial values for the unspecified variables from a standard normal distribution, which is not guaranteed to work well for all problems. With the default setting `mode='direct'`, the `prob.solve()` method first checks the feasibility of the current initial point. If the initial point is infeasible, then algorithm 3.2 will be used to try to find a feasible starting point, based on the current values of the variables, before launching the ACS procedure. We implement a scale-aware version of the stopping criterion (3.6). Let $u = f_0(x^{(k+1)}, y^{(k)})$ and $v = f_0(x^{(k+1)}, y^{(k+1)})$ denote the objective values of the two subproblems in one iteration. The ACS procedure terminates when

$$|u - v| \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max\{|u|, |v|\}. \tag{5.1}$$

The argument `abs_tol` to `prob.solve()` sets ϵ_{abs} and defaults to 10^{-6}. The argument `rel_tol` sets ϵ_{rel} and also defaults to 10^{-6}. Setting `rel_tol` to zero gives a criterion involving only the absolute tolerance.

Selecting the penalty method. Setting `mode='penalty'` in `prob.solve()` directly solves the relaxed biconvex problem (3.11) using the penalty ACS procedure (algorithm 3.3) from the current variable values, thereby permitting an infeasible starting point. If initial values are not specified, random values are generated as with the direct method. The option `nu` sets the positive penalty parameter ν in (3.11), and `slack_tol`, which defaults to 10^{-6}, sets the tolerance for the final total slack. Note that although the problem (3.11) is always feasible, there is no guarantee that the final point returned by `prob.solve()` is feasible for the original biconvex problem (2.1). To monitor the feasibility progress of the algorithm, the value of the total slack $\mathbf{1}^T s + \|t\|_1$ is reported after each ACS iteration. In the case that the final point is still infeasible with total slack exceeding `slack_tol`, the user may try to increase the value of `nu` and resolve the same problem object.

Solution statuses. After the ACS procedure terminates normally, the DBCP termination status is stored in `prob.status` and takes one of the following four values:

- `converged`: The stopping criterion (5.1) is satisfied, and if the penalty method is used, the final total slack does not exceed `slack_tol`.
- `converged_with_slack`: The stopping criterion (5.1) is satisfied, but the final total slack exceeds `slack_tol`.
- `converged_inaccurate`: The specified maximum number of ACS iterations is reached before the stopping criterion is satisfied, and if the penalty method is used, the final total slack does not exceed `slack_tol`.
- `converged_inaccurate_with_slack`: The maximum number of ACS iterations is reached before the stopping criterion is satisfied, and the final total slack exceeds `slack_tol`.

Note that for the direct method, `prob.status` can only take the two values without the `_with_slack` suffix.

Verification of biconvexity. When a biconvex optimization problem is specified as a `BiconvexProblem` instance, `dbcp` will automatically verify the biconvexity of the objective function and all constraints according to the DBCP ruleset introduced in Sect. 4. The user can check whether a problem is DBCP-compliant by calling `prob.is_dbcp()`, which returns `True` if the problem is a valid DBCP biconvex problem, and `False` otherwise. If a user is trying to solve a non-DBCP-compliant biconvex problem, `dbcp` will raise an error directly after the `prob.solve()` method is called.

Generalized inequality constraints. Lastly, we should mention that although all the previous discussion about solving and finding a feasible initial point for biconvex problems is based on standard inequality constraints as shown in (2.1), the `dbcp` package is implemented in a way that natively supports biconvex problems with generalized inequality constraints, such as second-order cone constraints and positive semidefinite constraints. Interested readers may refer to Sect. A for more theoretical details about how these constraints are handled in the backend, while practitioners can directly use the standard CVXPY syntax for specifying generalized inequality constraints when defining biconvex problems using `dbcp`. The appearance of such constraints will be automatically detected by `dbcp`, and the appropriate adaptations will be made to the solution procedure without any extra effort from the user.

6 Examples

In this section, we present several numerical examples using the `dbcp` package to specify and solve biconvex optimization problems that appear frequently in practice. We only show snippets of the code for defining and solving the biconvex problems, while omitting the data generation and result visualization parts for brevity. Interested readers can find the full code examples at

https://github.com/dxogrp/dbcp/tree/main/examples.

6.1 Nonnegative Matrix Factorization

We start with a basic nonnegative matrix factorization problem. Suppose that we are given a matrix $A \in \mathbf{R}^{m\times n}$, and are interested in finding two nonnegative matrices $X \in \mathbf{R}^{m\times k}$ and $Y \in \mathbf{R}^{k\times n}$ such that $A \approx XY$. This can be formulated as the following biconvex optimization problem:

$$\begin{array}{ll} \text{minimize} & \|XY - A\|_F^2 \\ \text{subject to} & X_{ij} \geq 0, \quad i = 1, \ldots, m, \quad j = 1, \ldots, k \\ & Y_{ij} \geq 0, \quad i = 1, \ldots, k, \quad j = 1, \ldots, n \end{array}$$

with variables X and Y. To specify this problem using `dbcp`, one may use the following code:

```
X = cp.Variable((m, k), nonneg=True)
Y = cp.Variable((k, n), nonneg=True)

obj = cp.Minimize(cp.sum_squares(X @ Y - A))
prob = BiconvexProblem(obj, [X], [Y])
```

In this example, we set $m = 5$, $n = 10$, and $k = 5$, generate two factor matrices with entries sampled uniformly from $[0, 1)$, and form A as their product. After calling `prob.solve()`, we obtain a final point with an objective value around 6.09×10^{-6}, which indicates that the original matrix A is well approximated by the product of the recovered nonnegative matrices X and Y.

6.2 Bilinear Logistic Regression

We consider a bilinear logistic regression problem for binary classification [21]. Suppose that we are given a dataset (X_i, y_i), $i = 1, \ldots, m$, where each sample consists of a feature matrix $X_i \in \mathbf{R}^{n\times k}$ and a binary label $y_i \in \{0, 1\}$. Our goal is to construct a bilinear classifier $\hat{y} = 1$ if $\mathbf{tr}(U^T X V) > 0$, and $\hat{y} = 0$ otherwise, where $U \in \mathbf{R}^{n\times r}$ and $V \in \mathbf{R}^{k\times r}$ are the bilinear logistic regression coefficients with a predefined (maximum) rank r, and $\mathbf{tr}(M)$ denotes the trace of some square matrix M. To fit a bilinear logistic regression model to the dataset, we would like to solve the following bilinear maximum likelihood estimation problem:

$$\text{maximize } \sum_{i=1}^{m} y_i \,\mathbf{tr}(U^T X_i V) - \log(1 + \exp(\mathbf{tr}(U^T X_i V)))$$

with variables U and V. This problem can be specified using `dbcp` as follows:

```
U = cp.Variable((n, r))
V = cp.Variable((k, r))

obj = cp.Maximize(sum(
    y * cp.trace(U.T @ X @ V)
    - cp.logistic(cp.trace(U.T @ X @ V))
    for X, y in zip(Xs, ys)))
prob = BiconvexProblem(obj, [U], [V])
```

In this example, we set $m = 300$, $n = 20$, $k = 10$, and $r = 5$. The dataset (X_i, y_i), $i = 1, \ldots, m$, is generated synthetically via the `make_classification` function from the scikit-learn library [53]. After calling `prob.solve()` with the proximal regularization weight `lbd=0.1` and the termination threshold `abs_tol=1e-4`, we obtain a final point with an objective value -1.44×10^{-3}.

6.3 *k*-Means Clustering

Suppose that we are given a set of data points $x_i \in \mathbf{R}^n$, $i = 1, \ldots, m$, and we would like to cluster them into k groups, using the k-means clustering method. This problem can be formulated as the following biconvex optimization problem [75]:

$$\begin{array}{ll} \text{minimize} & \sum_{i=1}^{m} z_i^T (\|\bar{x}_1 - x_i\|_2^2, \ldots, \|\bar{x}_k - x_i\|_2^2) \\ \text{subject to} & 0 \preceq z_i \preceq \mathbf{1}, \quad \mathbf{1}^T z_i = 1, \quad i = 1, \ldots, m \end{array} \tag{6.1}$$

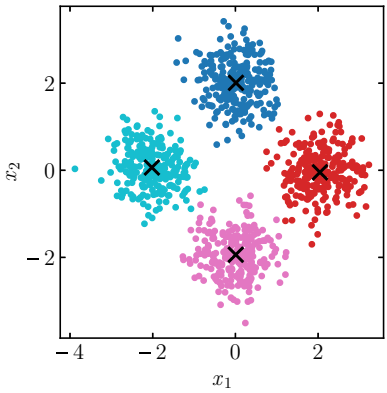


Fig. 3 Results of the k-means clustering example

with variables $\bar{x}_i \in \mathbf{R}^n$, $i = 1, \ldots, k$, and $z_i \in \mathbf{R}^k$, $i = 1, \ldots, m$.

We can interpret the variables in the problem (6.1) as follows: The variables $\bar{x}_1, \ldots, \bar{x}_k$ represent the cluster centroids, and each variable z_i is a soft assignment vector for the data point x_i, where the jth entry of z_i indicates the probability that the sample x_i belongs to cluster j. Then, the objective function in (6.1) represents the total within-cluster variance, which we would like to minimize. This formulation relaxes the original k-means problem by allowing each z_i to be a soft rather than a hard assignment. However, the relaxation is tight, since for fixed centroids, the objective is minimized by assigning each data point entirely to a nearest centroid (provided there is no tie in the distances). Therefore, the relaxed problem (6.1) always admits a hard assignment optimum and has the same optimal value as the original k-means problem.

To specify the problem (6.1) using `dbcp`, one may use the following code:

```
xbars = cp.Variable((k, n))
zs = cp.Variable((m, k), nonneg=True)

d = cp.sum_squares(xs[:, None, :] - xbars[None, :, :],
    axis=2)
obj = cp.Minimize(cp.sum(cp.multiply(zs, d)))
constr = [zs <= 1, cp.sum(zs, axis=1) == 1]
prob = BiconvexProblem(obj, [xbars], [zs], constr)
```

We generate a synthetic dataset of $m = 1000$ points in $\mathbf{R}^2$ using the `make_blobs` function of scikit-learn, and set the number of clusters $k = 4$ with the ground truth centroids $(0, 2)$, $(0, -2)$, $(2, 0)$, and $(-2, 0)$. The k-means clustering results based on the formulation (6.1) are shown in Fig. 3. After `prob.solve()` terminates, the displayed hard cluster label c_i of each data point is obtained from the optimized soft assignment vector z_i according to

$$c_i = \underset{j \in \{1,\ldots,k\}}{\operatorname{argmax}} z_{ij}, \quad i = 1, \ldots, m,$$

where the ties are resolved by choosing the smallest maximizing index. The colors indicate these cluster labels $c_1, \ldots, c_m$, and the black crosses represent the optimized centroids $\bar{x}_1, \ldots, \bar{x}_k$.

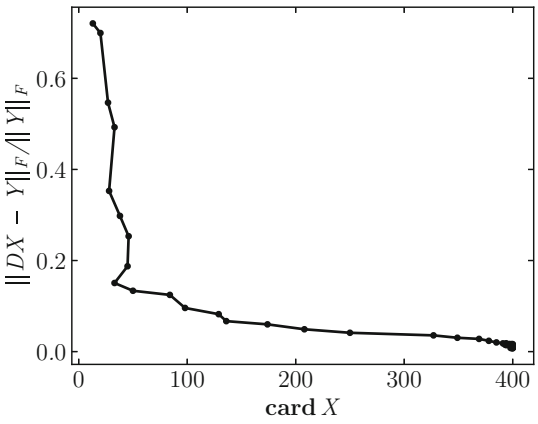


Fig. 4 Trade-off curve of the sparse dictionary learning example

6.4 Dictionary Learning

We consider the sparse dictionary learning problem [4], which aims to find a dictionary matrix $D \in \mathbf{R}^{m\times k}$ and a sparse code matrix $X \in \mathbf{R}^{k\times n}$, such that the data matrix $Y \in \mathbf{R}^{m\times n}$ can be well approximated by their product DX, while the matrix X is sparse and the matrix D has bounded Frobenius norm. The dictionary learning problem can be formulated as the following biconvex optimization problem:

$$\begin{array}{l}\text{minimize } \|DX - Y\|_F^2 + \alpha\|X\|_1 \\ \text{subject to } \|D\|_F \leq \beta\end{array} \tag{6.2}$$

with variables D and X, where $\alpha > 0$ is the sparsity regularization parameter, and $\beta > 0$ is the bound on the Frobenius norm of the dictionary matrix. To specify this problem using `dbcp`, one may use the following code:

```
D = cp.Variable((m, k))
X = cp.Variable((k, n))

obj = cp.Minimize(
    cp.sum_squares(D @ X - Y) + alpha * cp.norm1(X)
)
prob = BiconvexProblem(
    obj, [D], [X], [cp.norm(D, 'fro') <= beta]
)
```

In this example, we set $m = 10$, $n = 20$, $k = 20$, $\beta = 1$, and generate the data matrix $Y \in \mathbf{R}^{m\times n}$ from the standard normal distribution. The problem (6.2) is then solved for different values of the sparsity regularization parameter α ranging from 10^{-5} to 1. For each value of α, the values of the problem variables D and X are reinitialized randomly before `prob.solve()` is called. Figure 4 shows the trade-off curve between the relative approximation error $\|DX - Y\|_F/\|Y\|_F$ and the cardinality (*i.e.*, the number of nonzero entries) of X. We should note that this curve is not a Pareto frontier, as some plotted points are Pareto-dominated by others. The ACS procedure is a local method and need not return a global solution for each value of α. Moreover,

the returned point can depend on the random initialization. The optimization problem uses $\|X\|_1$ as a surrogate for sparsity, whereas the horizontal axis shows the cardinality of X, computed here as the number of entries whose absolute values are at least 10^{-3}. Since the ℓ_1-norm also depends on the magnitudes of the nonzero entries, even a global solution of problem (6.2) need not be Pareto optimal in terms of approximation error and cardinality.

6.5 Blind Deconvolution

Blind deconvolution is a technique used to recover some sharp signal or image from a blurred observation when the blur itself is unknown [16]. It jointly estimates the original signal and the blur kernel, with some prior knowledge about their structures. Suppose that we are given a data vector $d \in \mathbf{R}^{m+n-1}$, which is the convolution of an unknown sparse signal $x \in \mathbf{R}^n$ and an unknown smooth vector $y \in \mathbf{R}^m$ with bounded ℓ_∞-norm (*i.e.*, bounded largest absolute entry). Additionally, we have the prior knowledge that both vectors x and y are nonnegative. The corresponding blind deconvolution problem can be formulated as the following biconvex optimization problem:

$$\begin{array}{ll} \text{minimize} & \|x \otimes y - d\|_2^2 + \alpha_{\text{sp}}\|x\|_1 + \alpha_{\text{sm}}\|Dy\|_2^2 \\ \text{subject to} & x \succeq 0, \quad y \succeq 0 \\ & \|y\|_\infty \leq \beta \end{array}$$

with variables x and y, where $\alpha_{\text{sp}}, \alpha_{\text{sm}} > 0$ are the regularization parameters for the sparsity of x and smoothness of y, respectively, and $\beta > 0$ is the bound on the ℓ_∞-norm of the vector y. The matrix $D \in \mathbf{R}^{(m-1)\times m}$ is the first-order difference operator, given by

$$D = \begin{bmatrix} 1 & -1 & & \\ & 1 & -1 & \\ & & \ddots & \ddots \\ & & & 1 \quad -1 \end{bmatrix},$$

so that Dy computes the vector of successive differences of y. The convolution $x \otimes y$ of the vectors x and y is given by

$$(x \otimes y)_k = \sum_{i+j=k+1} x_i y_j, \quad k = 1, \ldots, m+n-1.$$

To specify this problem using `dbcp`, one may use the following code:

```
x = cp.Variable(n, nonneg=True)
y = cp.Variable(m, nonneg=True)

obj = cp.Minimize(
    cp.sum_squares(convolve(x, y) - d)
    + alpha_sp * cp.norm1(x)
    + alpha_sm * cp.sum_squares(cp.diff(y)))
constr = [cp.norm(y, 'inf') <= beta]
prob = BiconvexProblem(obj, [x], [y], constr)
```

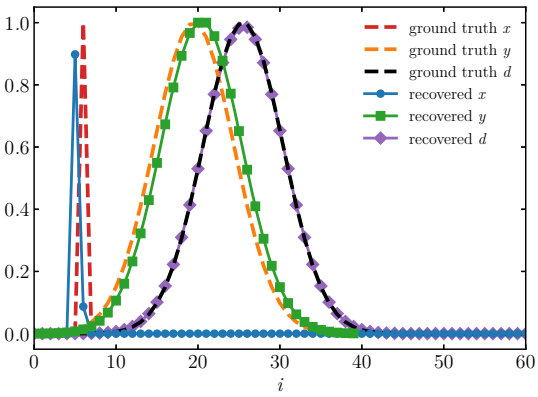


Fig. 5 Results of the blind deconvolution example

In this example, we consider the vectors $x \in \mathbf{R}^{120}$, $y \in \mathbf{R}^{40}$, and the ℓ_∞-norm bound $\beta = 1$. The ground truth vectors were generated according to a similar example used by Shen *et al.* [60]. The ground truth and the recovered signals after calling `prob.solve()` with regularization parameters $\alpha_{\text{sp}} = 0.1$ and $\alpha_{\text{sm}} = 0.2$ are shown in Fig. 5.

6.6 Fitting Input–Output Hidden Markov Models

We consider the problem of fitting a logistic input–output hidden Markov model (IO-HMM) to a dataset [39]. Suppose that we are given a dataset $(x(t), y(t)), t = 1, \ldots, m$, where each sample consists of an input feature vector $x(t) \in \mathbf{R}^n$ and an output label $y(t) \in \{0, 1\}$, generated from a K-state IO-HMM, according to the following procedure: Let $\hat{z}(t) \in \{1, \ldots, K\}, t = 1, \ldots, m$, be the state label of the IO-HMM with initial state distribution $p_{\text{init}} \in \mathbf{R}^K$ with $\mathbf{1}^T p_{\text{init}} = 1$ and transition matrix $P_{\text{tr}} \in \mathbf{R}^{K \times K}$ with $P_{\text{tr}}\mathbf{1} = \mathbf{1}$. At the time step t, the state label $\hat{z}(t)$ is sampled according to

$$\hat{z}(t) \sim \begin{cases} \text{Cat}(p_{\text{init}}), & t = 1 \\ \text{Cat}(p_{\hat{z}(t-1)}), & t > 1, \end{cases}$$

where the vector $p_{\hat{z}(t-1)} \in \mathbf{R}^K$ denotes the $\hat{z}(t-1)$th row of the matrix P_{tr}, and $\text{Cat}(p)$ denotes the categorical distribution with p being the vector of category probabilities. Then, given the feature vector $x(t) \in \mathbf{R}^n$, the output $y(t) \in \{0, 1\}$ of this IO-HMM at time step t is generated from a logistic model, *i.e.*,

$$\mathbf{prob}(y(t) = 1) = \frac{1}{1 + \exp(-x(t)^T \theta_{\hat{z}(t)})},$$

where $\theta_{\hat{z}(t)} \in \{\theta_1, \ldots, \theta_K\} \subseteq \mathbf{R}^n$ is the coefficient.

Given the dataset $(x(t), y(t))$, $t = 1, \ldots, m$, we are interested in recovering the transition matrix P_{tr}, the model parameters $\theta_1, \ldots, \theta_K$, and the unobserved state labels $\hat{z}(1), \ldots, \hat{z}(m)$. For each $t = 1, \ldots, m$, define the vector-valued expression $r(t) =$

$(r_1(t), \ldots, r_K(t)) \in \mathbf{R}^K$ componentwise by

$$r_k(t) = y(t)x(t)^T\theta_k - \log(1 + \exp(x(t)^T\theta_k)), \quad k = 1, \ldots, K.$$

Noticing that the transition matrix P_{tr} can be easily estimated from the state labels $\hat{z}(t)$, $t = 1, \ldots, m$, we consider the following biconvex optimization problem for fitting the IO-HMM [75]:

$$\begin{array}{ll} \text{minimize} & -\sum_{t=1}^m z(t)^T r(t) \\ & \quad +\alpha_\theta \sum_{k=1}^K \|\theta_k\|_2^2 + \alpha_z \sum_{t=1}^{m-1} D_{\text{kl}}(z(t), z(t+1)) \\ \text{subject to} & 0 \preceq z(t) \preceq \mathbf{1}, \quad \mathbf{1}^T z(t) = 1, \quad t = 1, \ldots, m \\ & \theta_k \in \mathcal{C}_k, \quad k = 1, \ldots, K, \end{array} \tag{6.3}$$

where the optimization variables are $\theta_k \in \mathbf{R}^n$, $k = 1, \ldots, K$, and $z(t) \in \mathbf{R}^K$, $t = 1, \ldots, m$. Note that the variable $z(t)$ is a soft assignment vector for the hidden state label $\hat{z}(t)$, where the kth entry of $z(t)$ indicates the probability of the state being k at the time step t, and $\hat{z}(t)$ can be estimated as the index of the largest entry of $z(t)$ after solving the problem (6.3). Each component of the problem (6.3) can be interpreted as follows: The first term in the objective function is the negative log-likelihood of the observed data under the IO-HMM model, given the state assignment probabilities $z(t)$, $t = 1, \ldots, m$, and the model parameters θ_k, $k = 1, \ldots, K$. The second term is a Tikhonov regularization on the model parameters θ_k, with the regularization parameter $\alpha_\theta > 0$. The third term is a temporal smoothness regularization on the state assignment probabilities, where $D_{\text{kl}}(p, q)$ denotes the Kullback–Leibler divergence between two probability distributions p and q, and $\alpha_z > 0$ is the corresponding regularization parameter. The constraints on the variables $z(t)$, $t = 1, \ldots, m$, ensure that they are valid probability distributions. The sets $\mathcal{C}_k \subseteq \mathbf{R}^n$, $k = 1, \ldots, K$, are closed nonempty convex sets that encode potential prior knowledge about the model parameters θ_k.

In this example, we generate $m = 1800$ data samples from an IO-HMM with $K = 3$ hidden states and input feature dimension $n = 2$. The feature vector for each sample is generated according to

$$x(t) \sim (\mathcal{U}(-5, 5),\ 1),$$

where $\mathcal{U}(a, b)$ denotes a uniform distribution over the interval $[a, b]$, and the second entry of $x(t)$ is always 1 to account for the bias term. The ground truth coefficients, initial state distribution, and transition matrix are given by

$$\theta_1 = (-1, 0), \quad \theta_2 = (2, 6), \quad \theta_3 = (2, -6),$$
$$p_{\text{init}} = (1, 0, 0), \quad P_{\text{tr}} = \begin{bmatrix} 0.95 & 0.025 & 0.025 \\ 0.025 & 0.95 & 0.025 \\ 0.025 & 0.025 & 0.95 \end{bmatrix}.$$

To fully specify the problem (6.3), it is assumed that we are given the following prior knowledge about the coefficients:

$$\theta_{1,1} \leq 0, \quad \theta_{2,1} \geq 0, \quad \theta_{3,1} \geq 0, \quad \theta_{2,2} \geq \theta_{3,2},$$

where $\theta_{i,j}$ denotes the jth entry of the vector θ_i. The corresponding code snippet is as follows:

```
thetas = cp.Variable((K, n))
zs = cp.Variable((m, K), nonneg=True)

rs = [
    -cp.multiply(ys, xs @ thetas[k])
    + cp.logistic(xs @ thetas[k])
    for k in range(K)
]
obj = cp.Minimize(
    cp.sum(cp.multiply(zs, cp.vstack(rs).T))
    + alpha_theta * cp.sum_squares(thetas)
    + alpha_z * cp.sum(cp.kl_div(zs[:-1], zs[1:])))
constr = [
    thetas[0][0] <= 0,
    thetas[1][0] >= 0,
    thetas[2][0] >= 0,
    thetas[1][1] >= thetas[2][1],
    zs <= 1, cp.sum(zs, axis=1) == 1
]

prob = BiconvexProblem(obj, [zs], [thetas], constr)
```

Note that for this example, we call the method `BiconvexProblem.solve()` with `mode='penalty'` to solve the relaxed biconvex problem (3.11) via the penalty ACS procedure (algorithm 3.3).

The problem (6.3) is then solved with regularization parameters $\alpha_\theta = 0.1$ and $\alpha_z = 2$. The arguments to the `prob.solve()` method are set to `mode='penalty'`, `nu=1e2`, `lbd=0.1`, and `abs_tol=1e-3`. The final total slack is around 4.45×10^{-8}, indicating that the returned point is feasible (within numerical roundoff error) for the original biconvex problem (6.3). The top panel of Fig. 6 shows the decoded state labels $\hat{z}(t) = \text{argmax}_{k\in\{1,\ldots,K\}} z_k(t)$, $t = 1, \ldots, m$, obtained from the optimized soft assignments, with ties resolved by the smallest maximizing index. The bottom panel shows the fitted response curves. The estimated transition matrix of the Markov chain is

$$\begin{bmatrix} 0.96 & 0.02 & 0.02 \\ 0.02 & 0.96 & 0.02 \\ 0.02 & 0.03 & 0.95 \end{bmatrix}.$$

7 Discussion

7.1 Conclusion

In this paper, we introduce DBCP, a DSL for specifying and solving biconvex optimization problems. When an optimization problem is compliant with the DBCP ruleset, its biconvex structure can be verified and the problem can be automatically split into two DCP-compliant convex subproblems, which are then solved using an ACS procedure. We implement these ideas in the open-source Python package `dbcp`, an extension

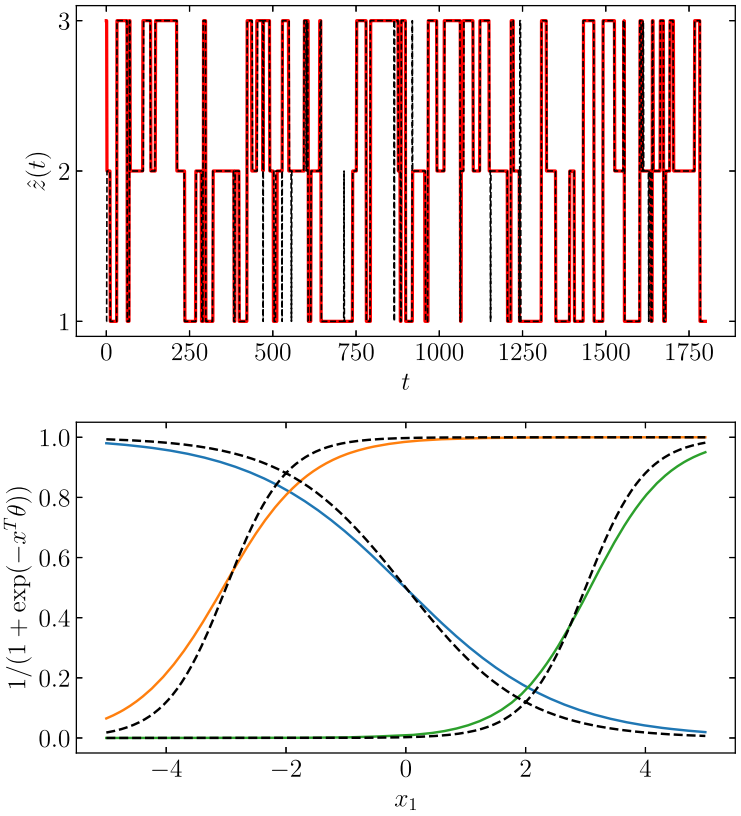


Fig. 6 Results of the IO-HMM example. The ground truth state labels (*top*) and the response curve of the output probability (*bottom*) are shown in black dashed lines, whereas the corresponding estimates are shown in solid lines

of CVXPY that allows users to express biconvex problems in a high-level, human-readable way while automatically handling the construction and solution of the convex subproblems.

We should note that DBCP cannot eliminate the solution difficulties intrinsic to biconvex optimization. Its ruleset is sufficient but not necessary, so not every biconvex formulation is recognized. For compliant problems, ACS does not generally guarantee global optimality, and its result can depend on initialization. What DBCP can do is to reduce the expert barrier in problem modeling and implementation, and to provide a convenient interface for users to explore different biconvex formulations and solution procedures.

The introduced examples show that models from several application areas can be expressed in compact, readable code close to their mathematical formulations. We hope this lowers the barrier to biconvex optimization in practice and encourages applications in new areas.

7.2 Comparison with DMCP

The DMCP framework [60] can be considered an early prototype for the DBCP framework we present here. In addition to being based on an early version of CVXPY and no longer maintained, it has a different modeling scope and implementation design.

First of all, DMCP is designed for multi-convex problems, while DBCP is designed for biconvex problems. The focus on two variable groups allows DBCP to draw directly on the more developed theoretical foundation for biconvex programming, including the partial optimality properties and ACS convergence results reviewed in Sects. 2 and 3; see also Wendell and Hurter Jr. [74] and Gorski *et al.* [33]. Secondly, as far as we know, most multi-convex problems that appear in practical applications can be formulated as biconvex problems by grouping variables appropriately. As a very basic example, the multi-convex problem

$$\begin{array}{l}\text{minimize } |z_1 z_2 + z_2 z_3 + z_3 z_4| \\ \text{subject to } z_1 + z_2 + z_3 + z_4 = 1\end{array}$$

with variables $z = (z_1, z_2, z_3, z_4) \in \mathbf{R}^4$ is multi-convex since, when any three variables are *fixed* [60], the objective is the absolute value of an affine function of the remaining variable, and the constraint is affine. However, this problem can be reformulated as a biconvex problem by grouping the variables as $x = (z_1, z_3) \in \mathbf{R}^2$ and $y = (z_2, z_4) \in \mathbf{R}^2$, which leads to the biconvex problem

$$\begin{array}{l}\text{minimize } \left| x^T \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} y \right| \\ \text{subject to } \mathbf{1}^T x + \mathbf{1}^T y = 1.\end{array}$$

Such a variable transformation allows each ACS step to update several variables simultaneously. When a compatible two-group representation is available, it is reasonable to expect that these joint updates may improve the efficiency and robustness of the solution procedure compared with treating each variable as a separate block as in the multi-convex case.

Regarding the implementation design, the DMCP framework registers a block coordinate descent *solver* for a standard CVXPY problem. In contrast, DBCP uses a specialized `BiconvexProblem` object. This object-oriented design allows the modeled biconvex problem and its components to be represented explicitly and inspected directly by a user. The two frameworks also handle initialization and solution procedures differently. The DMCP framework uses user specified or random initial values and introduces penalized slack variables throughout its block updates. By default, DBCP checks the initial point and, when necessary, attempts a separate feasibility search before applying ACS to the original, unrelaxed problem. This separates feasibility recovery from objective optimization, while the slack-penalized ACS is used as an optional alternative. We believe that our design choices fit better into the current CVXPY ecosystem, and we hope that they could provide the user with more flexibility and control over, *e.g.*, the problem modeling, solution process, and results inspection.

7.3 Comparison with DNLP

In parallel with the development of this work, the DNLP [15] framework was independently developed and released. We view DBCP and DNLP as complementary modeling options rather than competing replacements. Although their modeling scopes overlap, they expose different problem structures and use different local solution procedures. For example, the nonnegative matrix factorization problem presented in Sect. 6.1 can be specified using either framework. Several applications in the DNLP examples, including circle packing, path planning, sparse signal recovery, and phase retrieval, also appear among the earlier disciplined convex-concave programming examples [59]. This overlap illustrates that the same application can be approached through different disciplined modeling frameworks, whose rules and solution procedures remain distinct.

DNLP permits general smooth nonlinear expressions together with structured compositions of nonsmooth convex and concave functions. For a specified general optimization problem, it checks whether the problem satisfies the DNLP rules and, if so, canonicalizes it into a smooth nonlinear program and passes it to a supported nonlinear programming solver, such as IPOPT [71], KNITRO [14], COPT [29], and Uno [69, 70]. DBCP instead applies ACS to its two DCP-compliant convex subproblems using CVXPY-compatible convex optimization solvers. DBCP therefore does not introduce a dependency on a nonlinear programming solver. For users who already have the standard CVXPY convex-solving stack installed, this results in a lighter set of solver dependencies.

The initialization procedures of the two frameworks also differ. DNLP initializes auxiliary variables introduced during canonicalization and passes the complete nonlinear program, with user specified or generated initial values, to a nonlinear programming solver. By comparison, the direct method in DBCP uses the separate feasibility search workflow described above before applying ACS to the original problem. Since both frameworks use local solution procedures, they can be sensitive to initialization and may converge to different local optima. In the most general case, because neither framework can guarantee global optimality, there is not much we can say about the relative quality of the returned solutions.

From the perspective of application, DNLP applies to a broader range of nonlinear formulations, while DBCP provides an alternative when a useful biconvex decomposition is available and the user wishes to expose and exploit that structure. On the other hand, since both frameworks build on CVXPY variables, objectives, constraints, and expressions, when a formulation satisfies both rule systems, its objective and constraints can generally be reused by changing the problem construction and solve call. This makes it straightforward for users to try both approaches on modeled problems in their shared scope.

A Biconvex Problem With Generalized Inequality Constraints

Consider a biconvex optimization problem with generalized inequality constraints of the form

$$\begin{array}{ll} \text{minimize} & f_0(x, y) \\ \text{subject to} & f_i(x, y) \preceq_{\mathcal{K}_i} 0, \quad i = 1, \ldots, m \\ & h_i(x, y) = 0, \quad i = 1, \ldots, p, \end{array} \tag{A.1}$$

where $f_0 \colon \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$ is a biconvex objective, $h_i \colon \mathcal{X} \times \mathcal{Y} \to \mathbf{R}$, $i = 1, \ldots, p$, are biaffine equality constraint functions, and $f_i \colon \mathcal{X} \times \mathcal{Y} \to \mathbf{R}^{q_i}$, $i = 1, \ldots, m$, are biconvex inequality constraint functions with respect to proper cones $\mathcal{K}_i \subseteq \mathbf{R}^{q_i}$, $i = 1, \ldots, m$. The ACS procedure (algorithm 3.1) and the proximal regularizations given by (3.7) and (3.8) can be directly applied to the generalized inequality constrained biconvex problem (A.1) without modification. However, the initialization procedure (algorithm 3.2) and the penalty ACS procedure (algorithm 3.3) need to be slightly modified to accommodate the generalized inequality constraints. Specifically, the relaxed biconvex feasibility problem (3.10) is changed to

$$\begin{array}{ll} \text{minimize} & \mathbf{1}^T s + \|t\|_1 \\ \text{subject to} & s \succeq 0 \\ & f_i(x, y) \preceq_{\mathcal{K}_i} s_i e_{\mathcal{K}_i}, \quad i = 1, \ldots, m \\ & h_i(x, y) = t_i, \quad i = 1, \ldots, p, \end{array} \tag{A.2}$$

where $e_{\mathcal{K}_i} \succeq_{\mathcal{K}_i} 0$ is any positive element of the proper cone $\mathcal{K}_i$. For example, for a second-order cone $\mathcal{K} = \{(x, t) \in \mathbf{R}^q \mid \|x\|_2 \leq t\}$, we can choose $e_{\mathcal{K}} = (0, 1) \in \mathbf{R}^q$; for a positive semidefinite cone $\mathcal{K} = \mathbf{S}^q_+$, we can choose $e_{\mathcal{K}} = I \in \mathbf{R}^{q \times q}$, where I is the identity matrix. Similarly, the relaxed biconvex optimization problem (3.11) is changed to

$$\begin{array}{ll} \text{minimize} & f_0(x, y) + \nu(\mathbf{1}^T s + \|t\|_1) \\ \text{subject to} & s \succeq 0 \\ & f_i(x, y) \preceq_{\mathcal{K}_i} s_i e_{\mathcal{K}_i}, \quad i = 1, \ldots, m \\ & h_i(x, y) = t_i, \quad i = 1, \ldots, p. \end{array} \tag{A.3}$$

We implement the `dbcp` package in a way that these generalized inequality constraints are natively supported. When a user specifies a DBCP-compliant biconvex problem, `dbcp` will automatically detect the presence of generalized inequality constraints, then determine whether the problem formulations (A.2) and (A.3) should be used instead of (3.10) and (3.11), and finally, modify the initialization procedure (algorithm 3.2) and the penalty ACS procedure (algorithm 3.3) accordingly. From a user's perspective, all these modifications are made in the backend and no extra input is required.

Acknowledgements This work has been funded as part of BrainLinks-BrainTools, which is funded by the Federal Ministry of Economics, Science and Arts of Baden-Württemberg within the sustainability program for projects of the Excellence Initiative II, and CRC/TRR 384 "IN-CODE". The authors thank the anonymous reviewers for their constructive comments and suggestions, which helped improve the manuscript as well as the associated software.

Funding Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If

material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Agrawal, A., Boyd, S.: Disciplined quasiconvex programming. Optim. Lett. **14**(7), 1643–1657 (2020)
2. Agrawal, A., Diamond, S., Boyd, S.: Disciplined geometric programming. Optim. Lett. **13**(5), 961–976 (2019)
3. Agrawal, A., Verschueren, R., Diamond, S., Boyd, S.: A rewriting system for convex optimization problems. J. Control Decis. **5**(1), 42–60 (2018)
4. Aharon, M., Elad, M., Bruckstein, A.: K-SVD: an algorithm for designing overcomplete dictionaries for sparse representation. IEEE Trans. Signal Process. **54**(11), 4311–4322 (2006)
5. Al-Khayyal, F.A., Falk, J.E.: Jointly constrained biconvex programming. Math. Oper. Res. **8**(2), 273–286 (1983)
6. Ali, A., Kolter, J.Z., Diamond, S., Boyd, S.: Disciplined convex stochastic programming: a new framework for stochastic optimization, pp. 62–71. UAI (2015)
7. Aumann, R.J., Hart, S.: Bi-convexity and bi-martingales. Israel J. Math. **54**(2), 159–180 (1986)
8. Borwein, J.M.: Partially monotone operators and the generic differentiability of convex-concave and biconvex mappings. Israel J. Math. **54**(1), 42–50 (1986)
9. Boyd, S., Mattingley, J.: Branch and bound methods. Notes for EE364b, Stanford University **2006**, 07 (2007)
10. Boyd, S., Parikh, N., Chu, E., Peleato, B., Eckstein, J.: Distributed optimization and statistical learning via the alternating direction method of multipliers. Foundations and Trends® in Machine Learning **3**(1), 1–122 (2011)
11. Boyd, S., Vandenberghe, L.: Convex Optimization. Cambridge University Press, Cambridge (2004)
12. Burkholder, D.L.: A geometrical characterization of Banach spaces in which martingale difference sequences are unconditional. The Annals of Probability pp. 997–1011 (1981)
13. Burkholder, D.L.: Martingales and Fourier analysis in Banach spaces. In: Dold, A., Eckmann, B., (eds.:) Probability and Analysis, Lecture Notes in Mathematics, pp. 61–108. Springer (1986). Lectures given at the 1st 1985 Session of the Centro Internazionale Matematico Estivo (CIME) held at Varenna (Como), Italy, May 31–June 8, 1985
14. Byrd, R.H., Nocedal, J., Waltz, R.A.: KNITRO: An integrated package for nonlinear optimization. Large-Scale Nonlinear Optimization pp, pp. 35–59. (2006)
15. Cederberg, D., Zhang, W., Nobel, P., Boyd, S.: Disciplined nonlinear programming. (2026) arXiv:2606.02896
16. Chaudhuri, S., Velmurugan, R., Rameshan, R.: Blind Image Deconvolution: Methods and Convergence. Springer, Cham (2014)
17. Chu, W., Park, S.T.: Personalized recommendation on dynamic content using predictive bilinear models. In: Proceedings of the 18th International Conference on World Wide Web, pp. 691–700. (2009)
18. De Leeuw, J.: Block-relaxation algorithms in statistics. In: Information Systems and Data Analysis: Prospects—Foundations—Applications, pp. 308–324. Springer (1994)
19. Diamond, S., Boyd, S.: CVXPY: a Python-embedded modeling language for convex optimization. J. Mach. Learn. Res. **17**(83), 1–5 (2016)
20. Domahidi, A., Chu, E., Boyd, S.: ECOS: An SOCP solver for embedded systems. In: European Control Conference (ECC), pp. 3071–3076. (2013). https://doi.org/10.23919/ECC.2013.6669541
21. Dyrholm, M., Christoforou, C., Parra, L.C.: Bilinear discriminant component analysis. J. Mach. Learn. Res. **8**(5), (2007)
22. Falk, J.E., Soland, R.M.: An algorithm for separable nonconvex programming problems. Manage. Sci. **15**(9), 550–569 (1969)
23. Floudas, C.A.: Nonlinear and mixed-integer optimization: fundamentals and applications. Oxford University Press, Oxford (1995)
24. Floudas, C.A.: Deterministic Global Optimization: Theory, Methods and Applications, Nonconvex Optimization and its Applications, vol. 37. Springer Science & Business Media (2000)

25. Floudas, C.A., Visweswaran, V.: A global optimization algorithm (GOP) for certain classes of nonconvex NLPs – I. Theory. Comput. Chem. Eng. **14**(12), 1397–1417 (1990)
26. Floudas, C.A., Visweswaran, V.: Primal-relaxed dual global optimization approach. J. Optim. Theory Appl. **78**(2), 187–225 (1993)
27. Fosson, S.M.: A biconvex analysis for Lasso ℓ_1 reweighting. IEEE Signal Process. Lett. **25**(12), 1795–1799 (2018)
28. Fu, A., Narasimhan, B., Boyd, S.: CVXR: an R package for disciplined convex optimization. J. Stat. Softw. **94**, 1–34 (2020)
29. Ge, D., Huangfu, Q., Wang, Z., Wu, J., Ye, Y.: Cardinal Optimizer (COPT) user guide, (2022) arXiv:2208.14314
30. Geng, Z., Huang, L.: Robust stability of systems with both parametric and dynamic uncertainties. Syst. Control Lett. **39**(2), 87–96 (2000)
31. Geng, Z., Huang, L.: Robust stability of the systems with mixed uncertainties under the IQC descriptions. Int. J. Control **73**(9), 776–786 (2000)
32. Goh, K.C., Turan, L., Safonov, M.G., Papavassilopoulos, G.P., Ly, J.H.: Biaffine matrix inequality properties and computational methods. In: Proceedings of 1994 American Control Conference, vol. 1, pp. 850–855. IEEE (1994)
33. Gorski, J., Pfeuffer, F., Klamroth, K.: Biconvex sets and optimization with biconvex functions: A survey and extensions. Math. Methods Oper. Res. **66**(3), 373–407 (2007)
34. Goulart, P.J., Chen, Y.: Clarabel: An interior-point solver for conic programs with quadratic objectives, (2024). arXiv Preprint arXiv:2405.12762
35. Grant, M., Boyd, S.: CVX: MATLAB software for disciplined convex programming, version 2.1. http://cvxr.com/cvx (2014)
36. Grant, M., Boyd, S., Ye, Y.: Disciplined convex programming. In: Global Optimization: From Theory to Implementation, pp. 155–210. Springer (2006)
37. Hu, Y., Koren, Y., Volinsky, C.: Collaborative filtering for implicit feedback datasets. In: IEEE International Conference on Data Mining, pp. 263–272. IEEE (2008)
38. Javanmardi, H., Dehghani, M., Mohammadi, M., Vafamand, N.: Bilinear matrix inequality-based nonquadratic controller design for polytopic-linear parameter varying systems. Int. J. Robust Nonlinear Control **30**(17), 7655–7669 (2020)
39. Jha, A., Ashwood, Z.C., Pillow, J.W.: Active learning for discrete latent variable models. Neural Comput. **36**(3), 437–474 (2024)
40. Johnson, C.C.: Logistic matrix factorization for implicit feedback data. Adv. Neural. Inf. Process. Syst. **27**(78), 1–9 (2014)
41. Jouak, M., Thibault, L.: Directional derivatives and almost everywhere differentiability of biconvex and concave-convex operators. Math. Scand., 215–224 (1985)
42. Lawler, E.L., Wood, D.E.: Branch-and-bound methods: a survey. Oper. Res. **14**(4), 699–719 (1966)
43. Lee, D.D., Seung, H.S.: Learning the parts of objects by non-negative matrix factorization. Nature **401**(6755), 788–791 (1999)
44. Lee, J.M.: On Burkholder's biconvex-function characterization of Hilbert spaces. Proc. Am. Math. Soc. **118**(2), 555–559 (1993)
45. Levin, A., Weiss, Y., Durand, F., Freeman, W.T.: Understanding and evaluating blind deconvolution algorithms. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 1964–1971. IEEE (2009)
46. Lloyd, S.: Least squares quantization in PCM. IEEE Trans. Inf. Theory **28**(2), 129–137 (1982)
47. Löfberg, J.: YALMIP: A toolbox for modeling and optimization in MATLAB. In: Proceedings of the IEEE International Symposium on Computer Aided Control Systems Design, pp. 284–289. (2004)
48. Mishra, A., Dey, D.K., Chen, K.: Sequential co-sparse factor regression. J. Comput. Graph. Stat. **26**(4), 814–825 (2017)
49. Nocedal, J., Wright, S.J.: Numerical optimization. Springer, Cham (2006)
50. O'Donoghue, B., Chu, E., Parikh, N., Boyd, S.: Conic optimization via operator splitting and homogeneous self-dual embedding. J. Optim. Theory Appl. **169**(3), 1042–1068 (2016)
51. Paatero, P., Tapper, U.: Positive matrix factorization: a non-negative factor model with optimal utilization of error estimates of data values. Environmetrics **5**(2), 111–126 (1994)
52. Parikh, N., Boyd, S.: Proximal algorithms. Foundations and Trends® in Optimization **1**(3), 127–239 (2014)

53. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: machine learning in Python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
54. Powell, M.J.D.: On search directions for minimization algorithms. Math. Program. **4**(1), 193–201 (1973)
55. Rockafellar, R.T.: Convex Analysis. Princeton University Press, Princeton (1970)
56. Safonov, M.G., Goh, K.C., Ly, J.H.: Control system synthesis via bilinear matrix inequalities In: Proceedings of 1994 American Control Conference, vol. 1, pp. 45–49. IEEE (1994)
57. Schaller, M., Boyd, S.: Estimating price elasticity matrices. Operat. Res. Forum **7**, 87 (2026)
58. Schiele, P., Luxenberg, E., Boyd, S.: Disciplined saddle programming. Trans. Mach. Learning Res. (2024). https://openreview.net/forum?id=KhMLfEIoUm
59. Shen, X., Diamond, S., Gu, Y., Boyd, S.: Disciplined convex-concave programming. In: IEEE 55th Conference on Decision and Control (CDC), pp. 1009–1014. IEEE (2016)
60. Shen, X., Diamond, S., Udell, M., Gu, Y., Boyd, S.: Disciplined multi-convex programming. In: 29th Chinese Control and Decision Conference, pp. 895–900. IEEE (2017)
61. Shi, J.V., Xu, Y., Baraniuk, R.G.: Sparse bilinear logistic regression, (2014). arXiv Preprint arXiv:1404.4104
62. So, G., Mahajan, G., Dasgupta, S.: Convergence of online k-means. In: International conference on artificial intelligence and statistics, pp. 8534–8569. (2022). (PMLR)
63. Stellato, B., Banjac, G., Goulart, P., Bemporad, A., Boyd, S.: OSQP: an operator splitting solver for quadratic programs. Math. Program. Comput. **12**(4), 637–672 (2020)
64. Thibault, L.: Continuity of measurable convex and biconvex operators. Proc. Am. Math. Soc. **90**(2), 281–284 (1984)
65. Toker, O., Özbay, H.: On the NP-hardness of solving bilinear matrix inequalities and simultaneous stabilization with static output feedback. In: Proceedings of 1995 American control conference, vol. 4, pp. 2525–2526. IEEE (1995)
66. Udell, M., Horn, C., Zadeh, R., Boyd, S.: Generalized low rank models. Found. Trends® Mach. Learning **9**(1), 1–118 (2016)
67. Udell, M., Mohan, K., Zeng, D., Hong, J., Diamond, S., Boyd, S.: Convex optimization in Julia. In: Proceedings of the workshop for high performance technical computing in dynamic languages, pp. 18–28. (2014)
68. VanAntwerp, J.G., Braatz, R.D.: A tutorial on linear and bilinear matrix inequalities. J. Process Control **10**(4), 363–385 (2000)
69. Vanaret, C., Leyffer, S.: Implementing a unified solver for nonlinearly constrained optimization. Math. Progr. Comput. (2026)
70. Vanaret, C., Montoison, A.: Uno: a composable framework for nonlinearly constrained optimization. J. Open Sour. Soft. **11**(123), 10: 229 (2026)
71. Wächter, A., Biegler, L.T.: On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Math. Program. **106**, 25–57 (2006)
72. Warga, J.: Minimizing certain convex functions. J. Soc. Ind. Appl. Math. **11**(3), 588–593 (1963)
73. Wen, Z., Yin, W., Zhang, Y.: Solving a low-rank factorization model for matrix completion by a nonlinear successive over-relaxation algorithm. Math. Program. Comput. **4**(4), 333–361 (2012)
74. Wendell, R.E., Hurter, A.P., Jr.: Minimization of a non-separable objective function subject to disjoint constraints. Oper. Res. **24**(4), 643–657 (1976)
75. Zhu, H., Yan, S., Hoffmann, J., Boedecker, J.: Multi-convex programming for discrete latent factor models prototyping, (2025). arXiv Preprint arXiv:2504.01431

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.